



PROJETO E ANÁLISE DE ALGORITMOS

AULA 8 – ORDENAÇÃO EM TEMPO LINEAR

8.1 ORDENAÇÃO POR COMPARAÇÃO

- Vimos alguns algoritmos que podem ordenar n números no tempo $O(n \lg n)$. Enquanto a ordenação por intercalação e a ordenação por heap atingem esse limite superior no pior caso, o quicksort o atinge na média. Além disso, para cada um desses algoritmos, podemos produzir uma sequência de n números de entrada que faz o algoritmo ser executado no tempo $\Omega(n \lg n)$.
- Esses algoritmos compartilham uma propriedade interessante: a sequência ordenada que eles determinam **se baseia apenas em comparações** entre os elementos da entrada. Denominamos esses algoritmos **ordenações por comparação**. Todos os algoritmos de ordenação apresentados até aqui são ordenações por comparação.

8.1 ORDENAÇÃO POR COMPARAÇÃO

- Em uma ordenação por comparação, usamos **somente comparações** entre elementos para obter informações de ordem sobre uma sequência de entrada $\langle a_1, a_2, \dots, a_n \rangle$.
- Isto é, dados dois elementos a_i e a_j , executamos um dos testes $a_i < a_j$, $a_i \leq a_j$, $a_i = a_j$, $a_i \geq a_j$ ou $a_i > a_j$, para determinar sua ordem relativa.
- Não podemos inspecionar os valores dos elementos nem obter informações de ordem sobre eles de qualquer outro modo.

8.1 ORDENAÇÃO POR COMPARAÇÃO

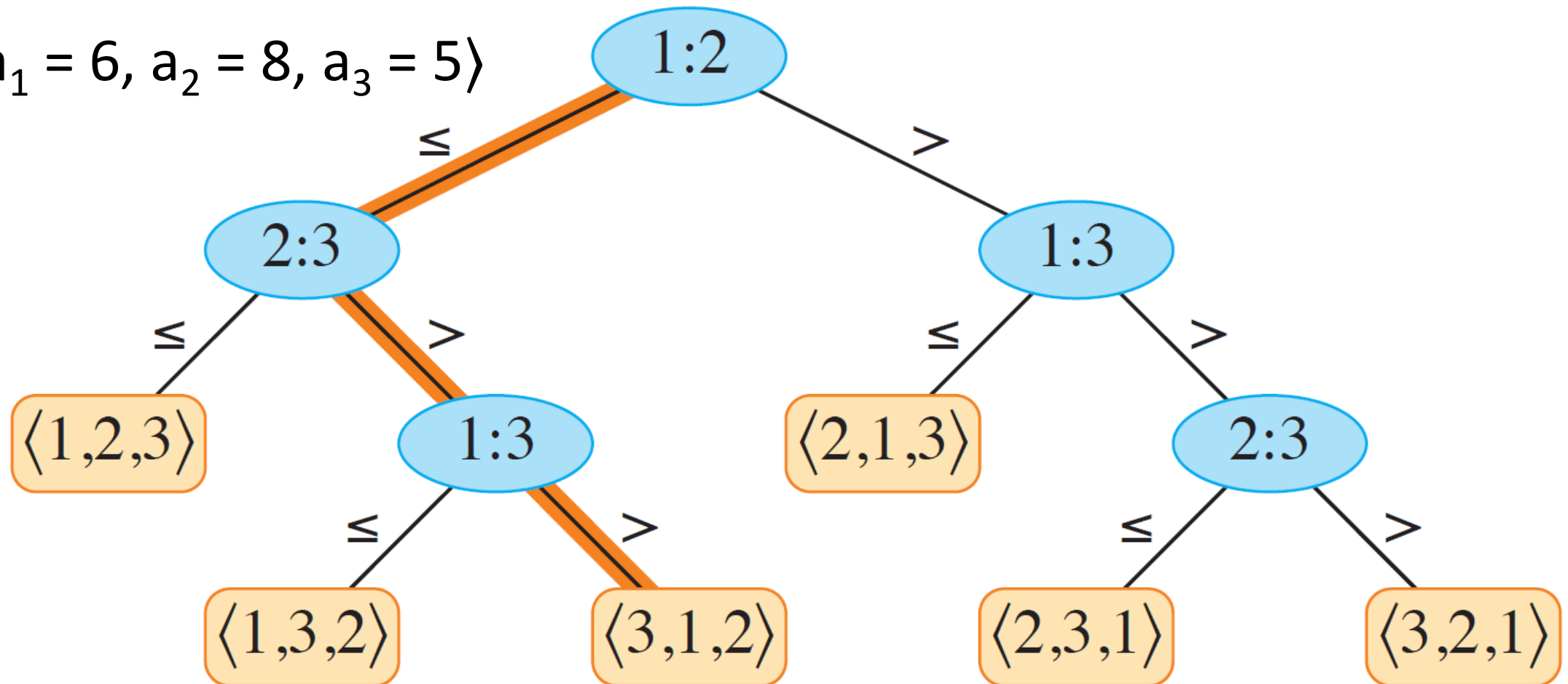
- Para um limite inferior, consideramos, sem perder a generalidade, que todos os elementos de entrada são distintos. Afinal, um limite inferior para elementos distintos é aplicado quando os elementos podem ou não ser distintos.
- Consequentemente, comparações da forma $a_i = a_j$ são inúteis, o que significa que podemos admitir que nenhuma comparação de igualdade exata é feita. Também observamos que as comparações $a_i \leq a_j$, $a_i \geq a_j$, $a_i > a_j$ e $a_i < a_j$ são equivalentes, já que produzem informações idênticas sobre a ordem relativa de a_i e a_j . Portanto, consideramos que todas as comparações têm a forma $a_i \leq a_j$.

8.1 ORDENAÇÃO POR COMPARAÇÃO

- Podemos imaginar as ordenações por comparação como **árvores de decisão**.
- Uma árvore de decisão é uma **árvore binária cheia** (cada nó é uma folha ou tem ambos os filhos) que representa as comparações entre elementos executadas por determinado algoritmo de ordenação operando sobre uma entrada de dado tamanho.
- Controle, movimentação de dados e todos os outros aspectos do algoritmo são ignorados.

8.1 ORDENAÇÃO POR COMPARAÇÃO

$\langle a_1 = 6, a_2 = 8, a_3 = 5 \rangle$



8.1 ORDENAÇÃO POR COMPARAÇÃO

- Um nó interno anotado como $i:j$ indica uma comparação entre a_i e a_j .
- Uma folha anotada como permutação $\langle \pi(1), \pi(2), \dots, \pi(n) \rangle$ indica a ordenação $a_{\pi(1)} \leq a_{\pi(2)} \leq \dots \leq a_{\pi(n)}$.
- O caminho com linha espessa indica as decisões tomadas durante a ordenação da sequência de entrada $\langle a_1 = 6, a_2 = 8, a_3 = 5 \rangle$. Partindo pela esquerda a partir da raiz, rotulada com 1:2, vemos que $a_1 \geq a_2$. Partindo pela direita a partir do nó rotulado com 2:3, vemos que $a_2 > a_3$. Partindo pela direita a partir do nó rotulado com 1:3, vemos que $a_1 > a_3$. Portanto, temos a ordenação $a_3 \geq a_1 \geq a_2$, conforme indicado na folha rotulada com $\langle 3, 1, 2 \rangle$.

8.1 ORDENAÇÃO POR COMPARAÇÃO

- A execução do algoritmo de ordenação corresponde a traçar um caminho simples desde a raiz da árvore de decisão até uma folha. Cada nó interno indica uma comparação $a_i \leq a_j$.
- Então, a subárvore da esquerda determina comparações subsequentes, já que sabemos que $a_i \leq a_j$ e a subárvore da direita determina comparações subsequentes quando $a_i > a_j$.

8.1 ORDENAÇÃO POR COMPARAÇÃO

- Quando chegamos a uma folha, o algoritmo de ordenação estabeleceu a ordenação $a_{\pi(1)} \leq a_{\pi(2)} \leq \dots \leq a_{\pi(n)}$.
- Como qualquer algoritmo de ordenação correto deve ser capaz de produzir cada permutação de sua entrada, cada uma das **$n!$ permutações** em n elementos deve aparecer como uma das folhas da árvore de decisão para uma ordenação por comparação ser correta.

8.1 ORDENAÇÃO POR COMPARAÇÃO

- O comprimento do caminho simples mais longo desde a raiz de uma árvore de decisão até qualquer uma de suas folhas acessíveis representa o número de comparações do pior caso que o algoritmo de ordenação correspondente executa.
- Consequentemente, o número de comparações do pior caso para dado algoritmo de ordenação por comparação é igual à **altura de sua árvore de decisão**.

8.1 ORDENAÇÃO POR COMPARAÇÃO

- Um limite inferior para as alturas de todas as árvores de decisão nas quais cada permutação aparece como uma folha acessível é, portanto, um limite inferior para o tempo de execução de qualquer algoritmo de ordenação por comparação.
- **É possível demonstrar que qualquer algoritmo de ordenação por comparação exige $\Omega(n \lg n)$ comparações no pior caso.**

8.2 ORDENAÇÃO POR CONTAGEM

- A ordenação por contagem (**Counting Sort**) supõe que cada um dos n elementos de entrada é um inteiro na faixa 0 a k , para algum inteiro k . Ela é executada em tempo $\Theta(n + k)$, de modo que, quando $k = O(n)$, a ordenação por contagem é executada no tempo $\Theta(n)$.
- A ordenação por contagem primeiro determina, para cada elemento de entrada x , o número de elementos menores ou iguais a x . Depois, usa essa informação para inserir o elemento x diretamente em sua posição no vetor de saída.

8.2 ORDENAÇÃO POR CONTAGEM

- Por exemplo, se 17 elementos são menores ou iguais a x , então x pertence à posição de saída 17. Temos de modificar ligeiramente esse esquema para lidar com a situação na qual vários elementos têm o mesmo valor, já que não queremos inserir todos eles na mesma posição.
- No procedimento ORDENA-POR-CONTAGEM, consideramos que a entrada é um vetor $A[1 : n]$, o tamanho n desse vetor e o limite k sobre os valores inteiros não negativos em A . Ele retorna sua saída ordenada no vetor $B[1 : n]$ e o vetor $C[0 : k]$ fornece armazenamento temporário adequado.

ORDENA-POR-CONTAGEM(A, n, k)

1 sejam B[1 : n] e C[0 : k] novos vetores

2 for i = 0 até k

3 C[i] = 0

4 for j = 1 até n

5 C[A[j]] = C[A[j]] + 1

6 // C[i] contém agora o número de elementos iguais a i.

7 for i = 1 até k

8 C[i] = C[i] + C[i - 1]

9 // C[i] contém agora o número de elementos menores ou iguais a i.

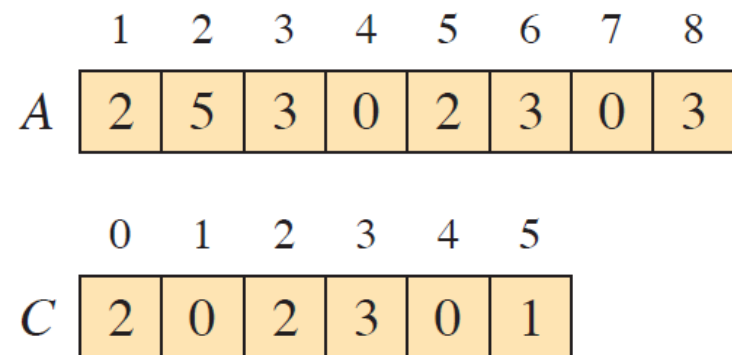
10 // Copia A para B, começando do final de A.

11 for j = n até 1

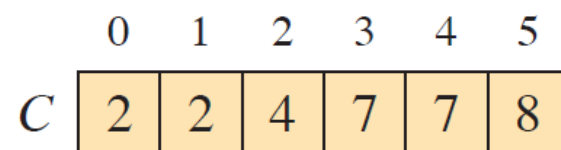
12 B[C[A[j]]] = A[j]

13 C[A[j]] = C[A[j]] - 1 //para lidar com valores duplicados

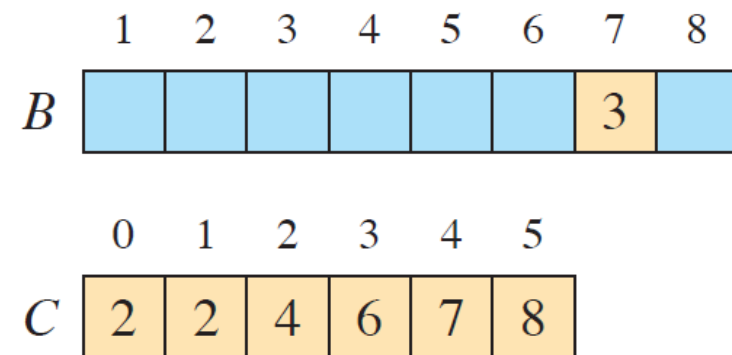
14 return B



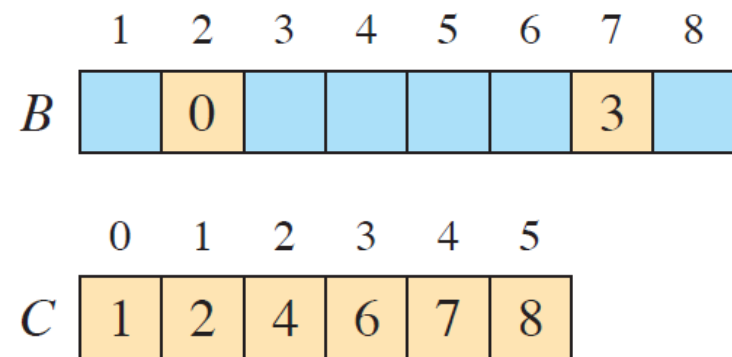
(a)



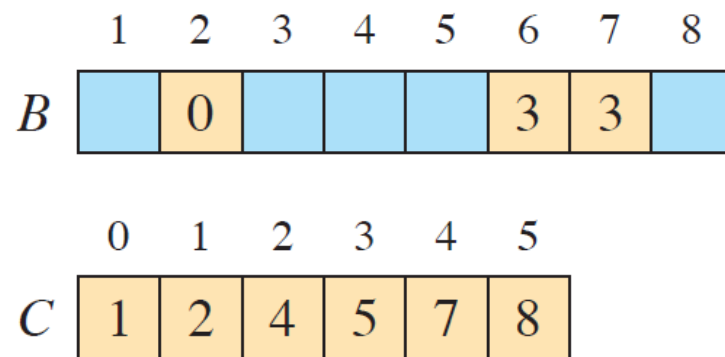
(b)



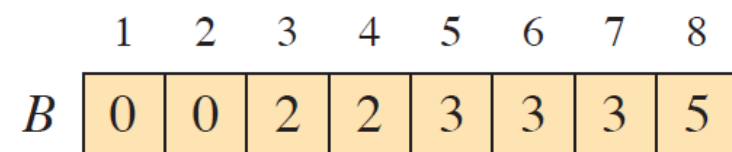
(c)



(d)



(e)



(f)

8.2 ORDENAÇÃO POR CONTAGEM

- Após o laço for das linhas 2–3 inicializar o vetor C com todos os valores iguais a zeros, o laço for das linhas 4–5 percorre o vetor A para inspecionar cada elemento da entrada. Se o valor de um elemento de entrada é i , incrementamos $C[i]$.
- Assim, depois da linha 5, $C[i]$ contém o número de elementos de entrada iguais a i para cada inteiro $i = 0, 1, \dots, k$. As linhas 7–8 determinam para cada $i = 0, 1, \dots, k$ quantos elementos de entrada são menores ou iguais a i , mantendo uma soma atualizada do vetor C.

8.2 ORDENAÇÃO POR CONTAGEM

- Finalmente, o laço for das linhas 11–13 percorre A novamente, mas na ordem inversa, para colocar cada elemento $A[j]$ em sua posição ordenada correta no vetor de saída B .
- Se todos os n elementos forem distintos, quando entrarmos pela primeira vez a linha 11, para cada $A[j]$, o valor $C[A[j]]$ será a posição final correta de $A[j]$ no vetor de saída, já que existem $C[A[j]]$ elementos menores ou iguais a $A[j]$.

8.2 ORDENAÇÃO POR CONTAGEM

- Como os elementos poderiam não ser distintos, o laço decrementa $C[A[j]]$ toda vez que insere um valor $A[j]$ no vetor B .
- Decrementar $C[A[j]]$ faz com que o elemento anterior em A com valor igual a $A[j]$, se existir algum, vá para a posição imediatamente anterior a $A[j]$ no vetor de saída B .
- O laço for das linhas 2–3 demora o tempo $\Theta(k)$, o laço for das linhas 4–5 demora o tempo $\Theta(n)$, o laço for das linhas 7–8 demora o tempo $\Theta(k)$ e o laço for das linhas 11–13 demora o tempo $\Theta(n)$. Assim, **o tempo total é $\Theta(k + n)$** . Na prática, normalmente usamos a ordenação por contagem quando temos $k = O(n)$, caso em que o tempo de execução é $\Theta(n)$.

8.2 ORDENAÇÃO POR CONTAGEM

- A ordenação por contagem supera o limite inferior de $\Omega(n \lg n)$ porque não é uma ordenação por comparação. De fato, **nenhuma comparação** entre elementos de entrada ocorre em qualquer lugar no código.
- Em vez disso, a ordenação por contagem utiliza os **valores reais** dos elementos para efetuar a **indexação em um vetor**. O limite inferior $\Omega(n \lg n)$ para ordenação não se aplica quando nos afastamos do modelo de ordenação por comparação.

8.2 ORDENAÇÃO POR CONTAGEM

- Uma propriedade importante da ordenação por contagem é ser **estável**: elementos com o mesmo valor aparecem no vetor de saída na mesma ordem em que aparecem no vetor de entrada.
- Isto é, ela desempata dois números segundo a regra de que qualquer número que **aparecer primeiro** no vetor de entrada aparecerá primeiro no vetor de saída.

8.2 ORDENAÇÃO POR CONTAGEM

- Normalmente, a propriedade de estabilidade só é importante quando **dados satélites** são transportados juntamente com o elemento que está sendo ordenado.
- A estabilidade da ordenação por contagem é importante por outra razão: a ordenação por contagem é usada frequentemente como uma sub-rotina na **ordenação digital**.

8.3 ORDENAÇÃO DIGITAL

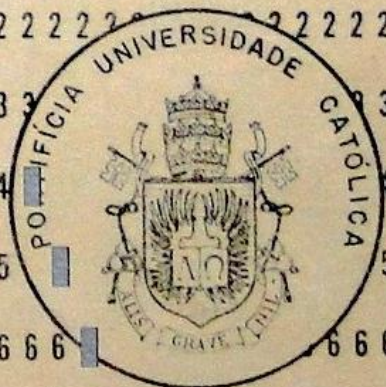
- Ordenação digital (Radix Sort) é o algoritmo usado pelas **máquinas de ordenação de cartões** que agora só encontramos em museus de computadores.
- Os cartões têm 80 colunas, e em cada coluna uma máquina pode fazer perfuração em uma das 12 posições. O ordenador pode ser “programado” mecanicamente para examinar determinada coluna de cada cartão em uma pilha e distribuir o cartão em uma das 12 caixas, dependendo do local em que foi perfurado.

11

15

CONTINUAÇÃO

PUC - R. MARQUÊS DE SÃO VICENTE, 209 - GAVEA



8.3 ORDENAÇÃO DIGITAL

- Então, um operador pode reunir os cartões caixa por caixa, de modo que aqueles que tenham a primeira posição perfurada fiquem sobre os cartões que tenham a segunda posição perfurada, e assim por diante.
- No caso de dígitos decimais, cada coluna utiliza apenas 10 posições (as outras duas posições são reservadas para codificar caracteres não numéricos). Então, um número de d dígitos ocuparia um campo de d colunas. Visto que o ordenador de cartões pode examinar apenas uma coluna por vez, o problema de ordenar n cartões em um número de d dígitos requer um algoritmo de ordenação.

8.3 ORDENAÇÃO DIGITAL

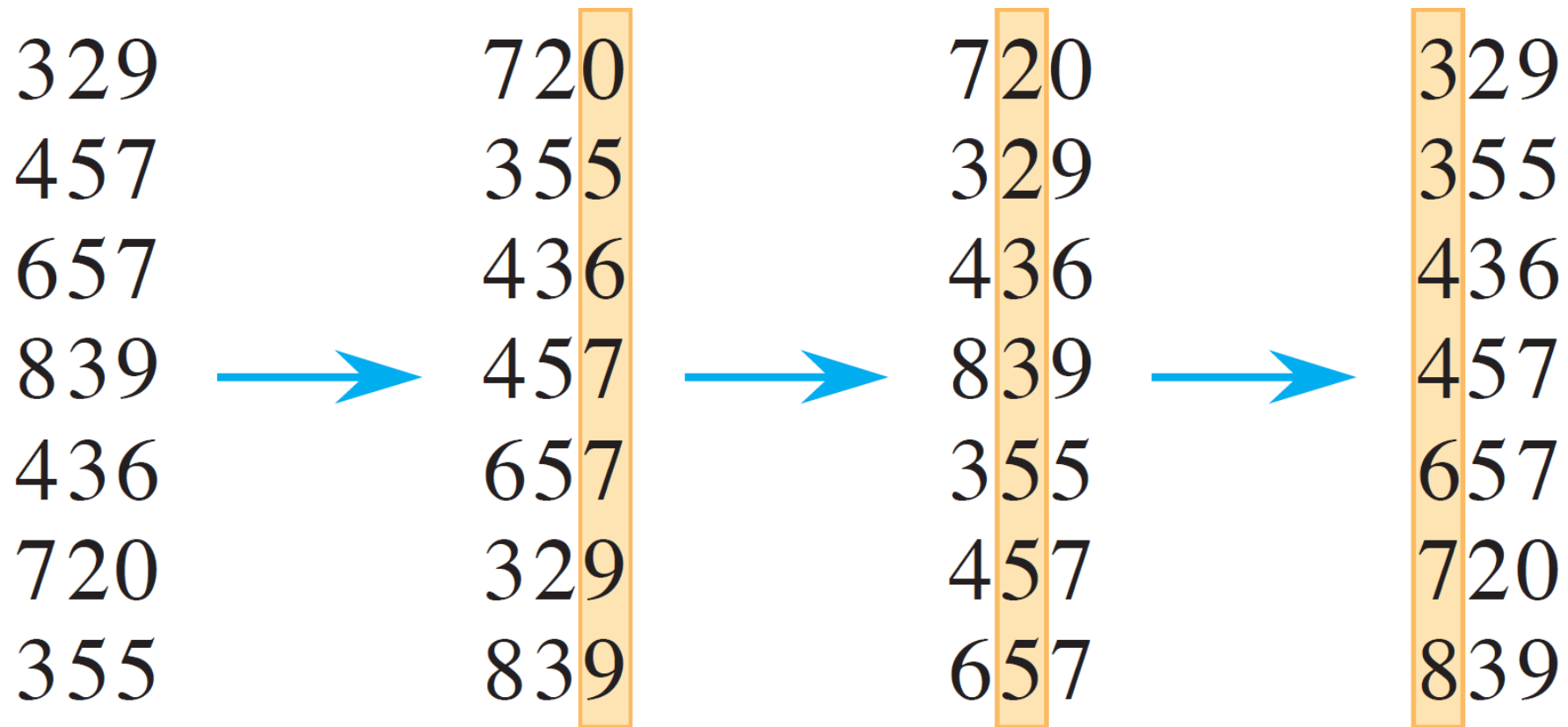
- Intuitivamente, poderíamos ordenar números sobre seu dígito mais significativo (mais à esquerda), ordenar cada uma das caixas resultantes recursivamente e depois combinar as pilhas em ordem.
- Infelizmente, como os cartões em nove das 10 caixas devem ser postos de lado para ordenar cada uma das caixas, esse procedimento gera muitas pilhas intermediárias de cartões que devem ser controladas.
- A ordenação digital resolve o problema da ordenação de cartões — contra a intuição normal — ordenando primeiro pelo dígito menos significativo.

8.3 ORDENAÇÃO DIGITAL

- O algoritmo combina os cartões em uma única pilha, sendo que os cartões na caixa 0 precedem os cartões na caixa 1, que precedem os cartões na caixa 2, e assim por diante.
- Então, ele ordena novamente a pilha inteira pelo segundo dígito menos significativo e recombina a pilha de maneira semelhante. O processo continua até que os cartões tenham sido ordenados por todos os d dígitos.
- Notável é que, nesse ponto, os cartões estão completamente ordenados pelo número de d dígitos. Assim, a ordenação exige apenas d passagens pela pilha.

8.3 ORDENAÇÃO DIGITAL

- Ordenação digital para uma pilha de sete números de três dígitos:



8.3 ORDENAÇÃO DIGITAL

- Em um computador típico, que é uma máquina sequencial de acesso aleatório, eventualmente, usamos ordenação digital para ordenar registros de informações chaveados por vários campos. Por exemplo, podemos querer ordenar datas por três chaves: ano, mês e dia.
- Podemos executar um algoritmo de ordenação com uma função de comparação que, conhecidas duas datas, compare anos e, se houver um empate, compare meses e, se ocorrer outro empate, compare dias.

8.3 ORDENAÇÃO DIGITAL

- Como alternativa, podemos ordenar as informações três vezes com uma ordenação estável: primeiro pelo dia (a parte “menos significativa”), em seguida pelo mês e, finalmente, pelo ano.
- O código para ordenação digital é direto. O procedimento ORDENA-POR-DÍGITO a seguir considera que cada elemento no vetor $A[1 : n]$ tem d dígitos, em que o dígito 1 é o dígito de ordem mais baixa e o dígito d é o dígito de ordem mais alta.

8.3 ORDENAÇÃO DIGITAL

ORDENA-POR-DÍGITO(A, n, d)

1 for i = 1 até d

**2 usar ordenação estável para ordenar o vetor
 A[1 : n] no dígito i**

8.3 ORDENAÇÃO DIGITAL

- **Lema:** dados n números de d dígitos nos quais cada dígito pode adotar até k valores possíveis, ORDENA-POR-DÍGITO ordenará corretamente esses números no tempo $\Theta(d(n + k))$ se a ordenação estável levar o tempo $\Theta(n + k)$.

8.3 ORDENAÇÃO DIGITAL

- **Prova:** A corretude da ordenação digital decorre por indução para a coluna que está sendo ordenada. A análise do tempo de execução depende da ordenação estável usada como algoritmo de ordenação intermediária. Quando cada dígito está no intervalo de 0 a $k - 1$ (de modo que pode adotar até k valores possíveis) e k não é muito grande, a ordenação por contagem é a escolha óbvia. Então, cada passagem sobre n números de d dígitos leva o tempo $\Theta(n + k)$. Há d passagens e, assim, o tempo total para ordenação digital é $\Theta(d(n + k))$.

8.3 ORDENAÇÃO DIGITAL

- Quando d é constante e $k = O(n)$, podemos executar a ordenação digital em tempo linear. De modo mais geral, temos alguma flexibilidade quanto aos modos de desmembrar cada chave em dígitos.

8.4 ORDENAÇÃO POR BALDE

- A ordenação por balde (bucket sort) supõe que a entrada é retirada de uma **distribuição uniforme** e tem tempo de execução do caso médio de $O(n)$.
- Assim como na ordenação por contagem, a ordenação por balde é rápida porque admite alguma coisa com relação à entrada.
- Enquanto a ordenação por contagem considera que a entrada consiste em inteiros contidos em um pequeno intervalo, a ordenação por balde admite que a entrada é gerada por um processo aleatório que distribui elementos de modo uniforme e independente no intervalo $[0, 1)$.

8.4 ORDENAÇÃO POR BALDE

- A ordenação por balde divide o intervalo $[0, 1)$ em n subintervalos de tamanhos iguais, ou baldes, e depois distribui os n números de entrada entre os baldes.
- Visto que as entradas são distribuídas de modo uniforme e independente por $[0, 1)$, **não esperamos que muitos números caiam em cada balde.**
- Para produzir a saída, simplesmente ordenamos os números em cada balde e depois percorremos os baldes em ordem, anotando os elementos contidos em cada um.

8.4 ORDENAÇÃO POR BALDE

- O procedimento ORDENA-POR-BALDE considera que a entrada é um vetor $A[1 : n]$ e que cada elemento $A[i]$ no vetor satisfaz $0 \leq A[i] < 1$.
- O código exige um vetor auxiliar $B[0 : n - 1]$ de listas encadeadas (baldes) e considera que existe um mecanismo para manter tais listas.

8.4 ORDENAÇÃO POR BALDE

ORDENA-POR-BALDE(A, n)

1 seja $B[0 : n - 1]$ um novo vetor

2 for $i = 0$ até $n - 1$

3 faça $B[i]$ uma lista vazia

4 for $i = 1$ até n

5 insira $A[i]$ na lista $B[\lfloor n \cdot A[i] \rfloor]$

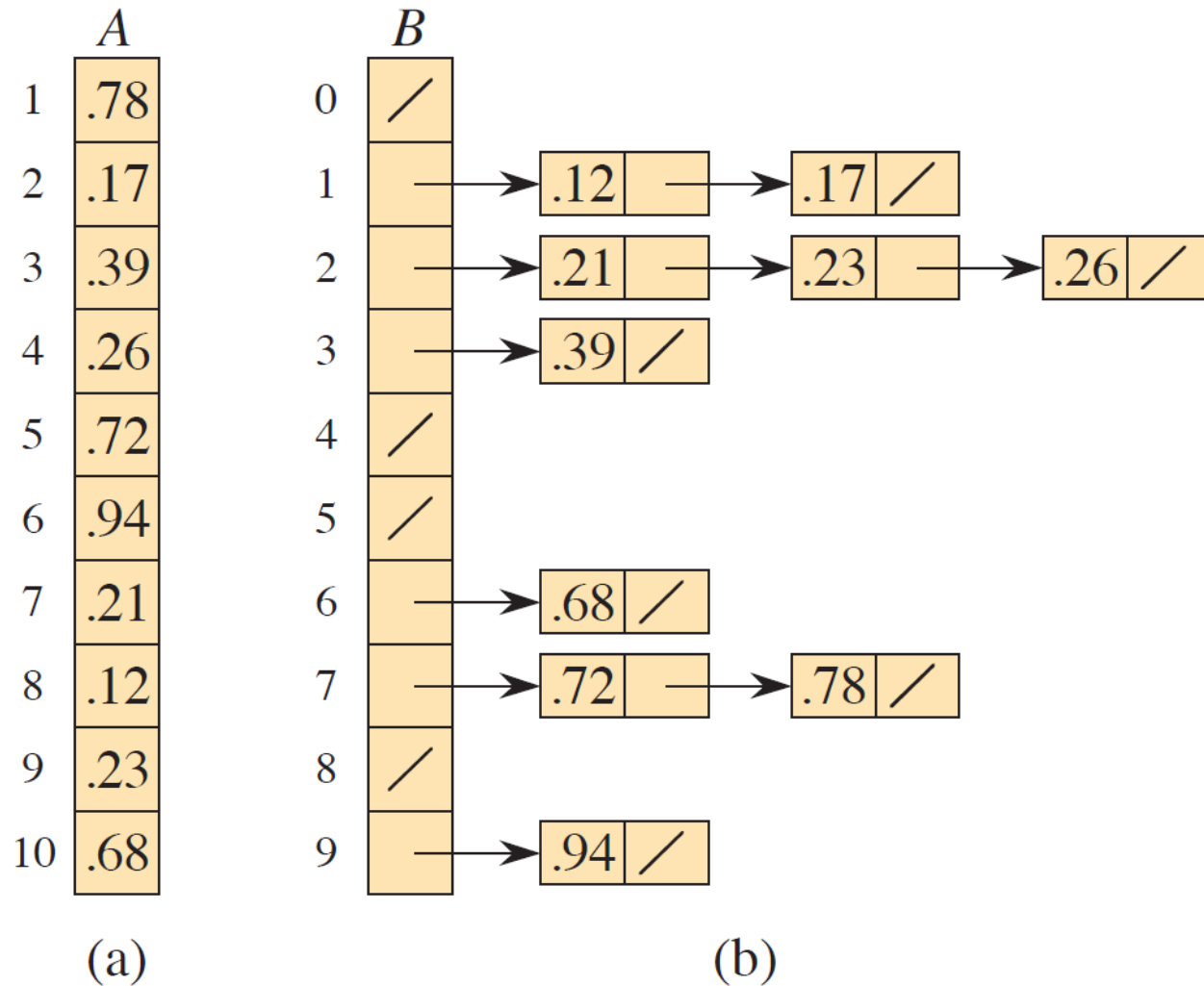
6 for $i = 0$ até $n - 1$

7 ordene a lista $B[i]$ com ordenação por inserção

8 concatene as listas $B[0], B[1], \dots, B[n - 1]$ em ordem

9 return as listas encadeadas

8.4 ORDENAÇÃO POR BALDE



8.4 ORDENAÇÃO POR BALDE

- Para ver que esse algoritmo funciona, considere dois elementos $A[i]$ e $A[j]$. Suponha, sem perda de generalidade, que $A[i] \leq A[j]$.
- Visto que $\lfloor n \cdot A[i] \rfloor \leq \lfloor n \cdot A[j] \rfloor$, o elemento $A[i]$ é inserido no mesmo balde que $A[j]$ ou em um balde com índice mais baixo. Se $A[i]$ e $A[j]$ entrarem no mesmo balde, o laço for das linhas 6–7 os colocará na ordem adequada. Se $A[i]$ e $A[j]$ entrarem em baldes diferentes, a linha 8 os colocará na ordem adequada. Portanto, a ordenação por balde funciona corretamente.